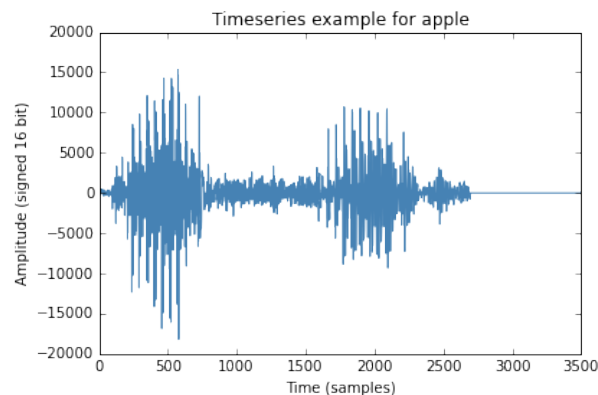


4. Markov Models and HMM

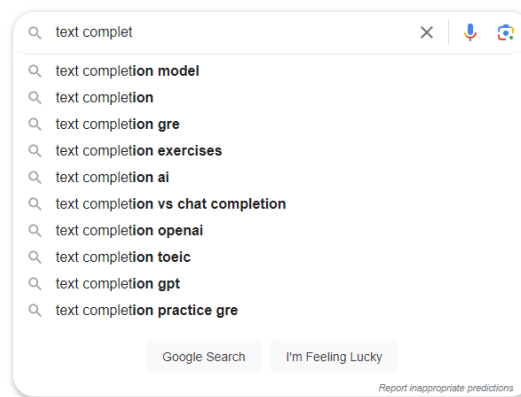
- 1 Markov Chains
- 2 Hidden Markov Models
- 3 Some Practice Problems
- 4 MLE Derivation (Supplementary)

Motivation

- Speech recognition: what is the last word spoken, given part of a sentence?



- Text generation: what is the most likely next character or word?



- Predict the next “state” based on the current “state”: this is a [Markov chain](#)!

Outline

- 1 Markov Chains
- 2 Hidden Markov Models
- 3 Some Practice Problems
- 4 MLE Derivation (Supplementary)

Markov Chains

- We consider discrete time sequence of random variables $x[0], x[1], \dots$. E.g., natural language processing, time series modeling, etc.
- Each $x[t] \in$ **state space** $\{1, 2, \dots, M\}$.
- Satisfies the Markov property:

$$p(x[t] \mid x[1], \dots, x[t-1]) = p(x[t] \mid x[t-1])$$

“The future is independent of the past, given the present.”

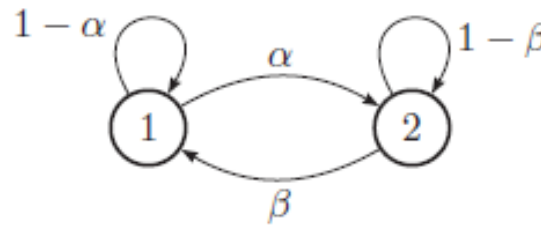
- $\mathbf{T}(i, j) = p_{x[t]|x[t-1]}(j \mid i) = \mathbb{P}(x[t] = j \mid x[t-1] = i)$ is called the transition probability. We consider only *homogeneous* MC here - transition probability does not depend on time index t .
- $\mathbf{T} = [\mathbf{T}(i, j)]_{i,j=1}^M$ is called the **transition matrix**.

Transition Matrix

- Each row of $\mathbf{T}$ sums to one:

$$\sum_{j=1}^M \mathbf{T}(i, j) = \sum_{j=1}^M \mathbb{P}(x[t] = j \mid x[t-1] = i) = 1.$$

- $\mathbf{T}$ is said to be a (row) **stochastic matrix**.
- Can use transition diagrams to represent a Markov chain:



$$\mathbf{T} = \begin{bmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{bmatrix}$$

Transition Matrix

- Suppose $x[0]$ has pdf $p_0 = [p_0(1), p_0(2), \dots, p_0(M)]$. Pdf of $x[1]$ is given by

$$\begin{aligned}
 p_1(i) &= \sum_{j=1}^M p(x[1] = i, x[0] = j) \\
 &= \sum_{j=1}^M p(x[1] = i \mid x[0] = j) p_0(j) \\
 &= \sum_{j=1}^M \mathbf{T}(j, i) p_0(j) \\
 &= (p_0 \mathbf{T})(i),
 \end{aligned}$$

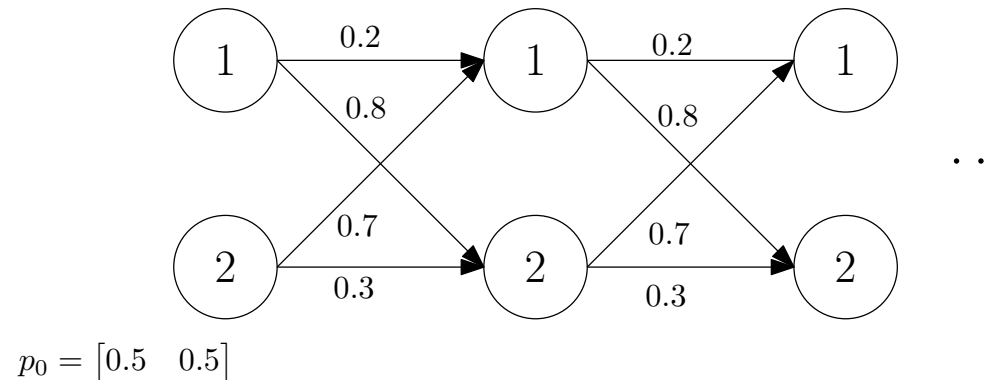
or $p_1 = p_0 \mathbf{T}$.

- Let p_t to denote the row vector $[p_t(1), \dots, p_t(M)]$. Similarly,

$$p_t = p_{t-1} \mathbf{T} = p_{t-2} \mathbf{T}^2 = \dots = p_0 \mathbf{T}^t.$$

Example

$$\mathbf{T} = \begin{bmatrix} 0.2 & 0.8 \\ 0.7 & 0.3 \end{bmatrix}$$



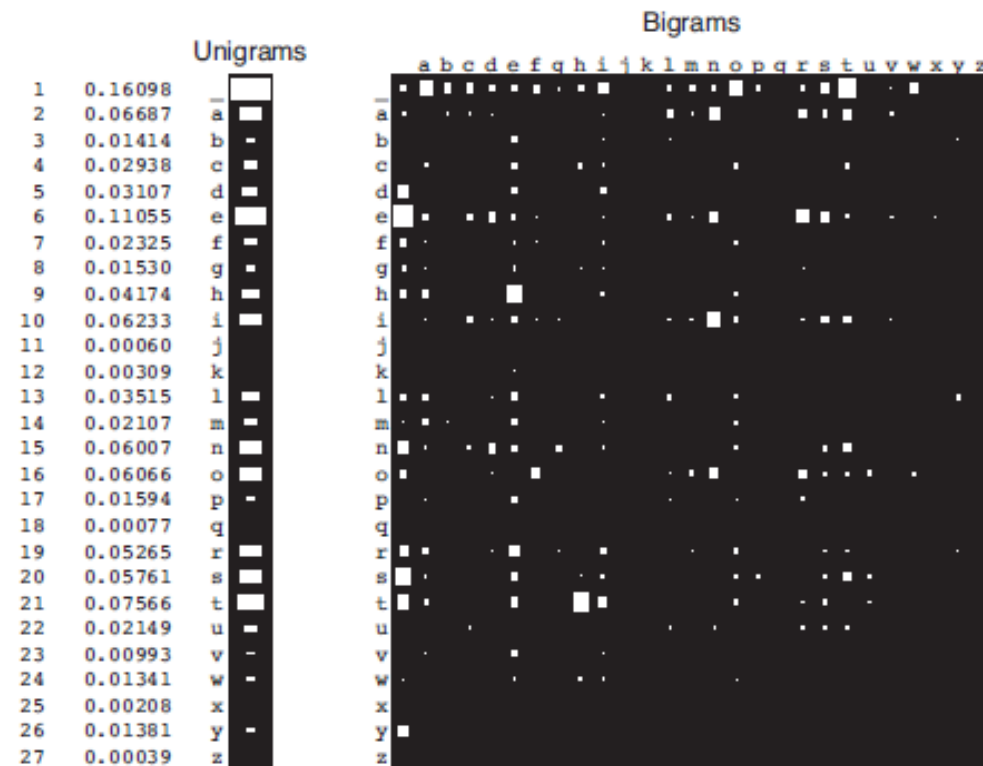
- $p_1 = p_0 \mathbf{T} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 0.2 & 0.8 \\ 0.7 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.45 & 0.55 \end{bmatrix}$
- $p_2 = p_1 \mathbf{T} = \begin{bmatrix} 0.45 & 0.55 \end{bmatrix} \begin{bmatrix} 0.2 & 0.8 \\ 0.7 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.475 & 0.525 \end{bmatrix}$
- What is the probability of the sequence $x[0] = 1, x[1] = 2, x[2] = 1$?

$$p(x[0] = 1, x[1] = 2, x[2] = 1) = p_0(1) \mathbf{T}(1, 2) \mathbf{T}(2, 1) = 0.5 \times 0.8 \times 0.7.$$

Application: Language Modeling

- Statistical language model: learn probability distribution of word sequences.
Applications:
 - ▶ Sentence completion: predict the next word given the previous words in a sentence.
 - ▶ Data compression: use shorter codewords to represent more frequently used strings to store the data.
 - ▶ Text classification: classify the topic or meaning of a document.
- State space: all words in a language, e.g., English.
- Unigram model: $p(x[t] = x)$ - marginal probability that a word is x .
- Bigram model: $p(x[t] \mid x[t - 1])$ - probability that a word is $x[t]$ depends on immediate previous word $x[t - 1]$.
- n-gram model: $p(x[t] \mid x[t - 1], x[t - 2], \dots, x[t - n + 1])$ - $x[t]$ depends on $n - 1$ immediate previous words

Application: Language Modeling



[M12]: Unigram and bigram counts for letters $\{a, \dots, z, -\}$ ($-$ stands for space) in Darwin's "On The Origin of Species". The 2D picture on the right is a Hinton diagram of the joint distribution. The size of the white squares is proportional to the value of the entry in the corresponding vector/matrix.

Application: PageRank

- A website is scored to be more **authoritative** if it is linked to by other **authoritative** websites.
- π_i : authoritative score of website i .
- $\mathbf{T}(j, i)$: probability that a user follows a link from website j to website i (if j does not link to i , we naturally have $\mathbf{T}(j, i) = 0$.)

$$\pi_i = \sum_j \mathbf{T}(j, i) \pi_j$$

- User visits form a Markov chain.

MLE

- Estimate prior π and transition matrix $\mathbf{T}$ from training data:

$$p(x[0], \dots, x[t] \mid \pi, \mathbf{T}) = \pi(x[0])\mathbf{T}(x[0], x[1])\mathbf{T}(x[1], x[2]) \cdots \mathbf{T}(x[t-1], x[t]).$$

- Suppose we observe n sequences:

$$\mathcal{D} = \{\mathbf{x}_1[0:t_1], \dots, \mathbf{x}_n[0:t_n]\},$$

and $\mathbf{x}_i[0:t_i] = (x_i[0], x_i[1], \dots, x_i[t_i])$ is a sequence of length $t_i + 1$.

- Note we can have sequences of different lengths, but we assume every data point follows same transition matrix $\mathbf{T}$.

MLE

$$\begin{aligned}
 \log p(\mathcal{D} \mid \boldsymbol{\pi}, \mathbf{T}) &= \sum_{i=1}^n \log \boldsymbol{\pi}(x_i[0]) + \sum_{i=1}^n \sum_{t=1}^{t_i} \log \mathbf{T}(x_i[t-1], x_i[t]) \\
 &= \sum_{x=1}^M N_x \log \boldsymbol{\pi}(x) + \sum_{x=1}^M \sum_{y=1}^M N_{xy} \log \mathbf{T}(x, y),
 \end{aligned}$$

where

$$\begin{aligned}
 N_x &= \sum_{i=1}^n \mathbf{1}\{x_i[0] = x\}, \\
 N_{xy} &= \sum_{i=1}^n \sum_{t=1}^{t_i} \mathbf{1}\{x_i[t-1] = x, x_i[t] = y\}.
 \end{aligned}$$

MLE

- Taking partial derivatives of the log likelihood and finding the roots, we obtain the MLE: for $x = 1, \dots, M$ and $y = 1, \dots, M$,

$$\hat{\pi}(x) = \frac{N_x}{n},$$
$$\hat{\mathbf{T}}(x, y) = \frac{N_{xy}}{\sum_{z=1}^M N_{xz}}.$$

- Problem: model may predict certain strings are impossible if data have zero count of certain states (this is a form of overfitting).
 - ▶ E.g., if we have 50,000 words in our vocabulary, a bigram model has 2.5 billion parameters. Likely to have certain word pairs not appearing in our training data.

Numerical Example

- State space = $\{1, 2\}$, $\mathcal{D} = \{\mathbf{x}_1[0 : 2], \mathbf{x}_2[0 : 1], \mathbf{x}_3[0 : 3]\}$
- $\mathbf{x}_1[0 : 2] = (1, 2, 1)$, $\mathbf{x}_2[0 : 1] = (2, 2)$, $\mathbf{x}_3[0 : 3] = (1, 1, 2, 1)$

$$\begin{aligned}
 \log p(\mathcal{D} \mid \boldsymbol{\pi}, \mathbf{T}) &= \log \boldsymbol{\pi}(1) + \log \boldsymbol{\pi}(2) + \log \boldsymbol{\pi}(1) \\
 &\quad + \log \mathbf{T}(1, 2) + \log \mathbf{T}(2, 1) \\
 &\quad + \log \mathbf{T}(2, 2) \\
 &\quad + \log \mathbf{T}(1, 1) + \log \mathbf{T}(1, 2) + \log \mathbf{T}(2, 1) \\
 &= 2 \log \boldsymbol{\pi}(1) + \log \boldsymbol{\pi}(2) \\
 &\quad + \log \mathbf{T}(1, 1) + 2 \log \mathbf{T}(1, 2) + 2 \log \mathbf{T}(2, 1) + \log \mathbf{T}(2, 2)
 \end{aligned}$$

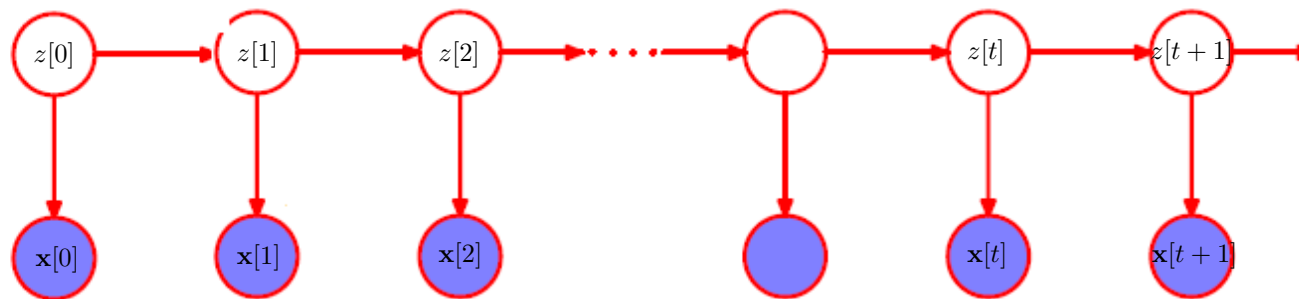
- $\hat{\boldsymbol{\pi}}(1) = 2/3$, $\hat{\boldsymbol{\pi}}(2) = 1/3$
- $\hat{\mathbf{T}} = \begin{bmatrix} 1/3 & 2/3 \\ 2/3 & 1/3 \end{bmatrix}$

Outline

- 1 Markov Chains
- 2 Hidden Markov Models
- 3 Some Practice Problems
- 4 MLE Derivation (Supplementary)

HMM

- A hidden Markov model (HMM) consists of:
 - ▶ A discrete state Markov chain with **hidden states or latent variables** $z[t] \in \{1, \dots, M\}$, $t = 0, 1, \dots$, with **initial pdf** π and **transition matrix** $\mathbf{T}$.
 - ▶ An observation model given by **emission probabilities** $p(\mathbf{x}[t] \mid z[t]) = p(\mathbf{x}[t] \mid \phi_{z[t]})$. Let $\phi = (\phi_1, \dots, \phi_M)$.

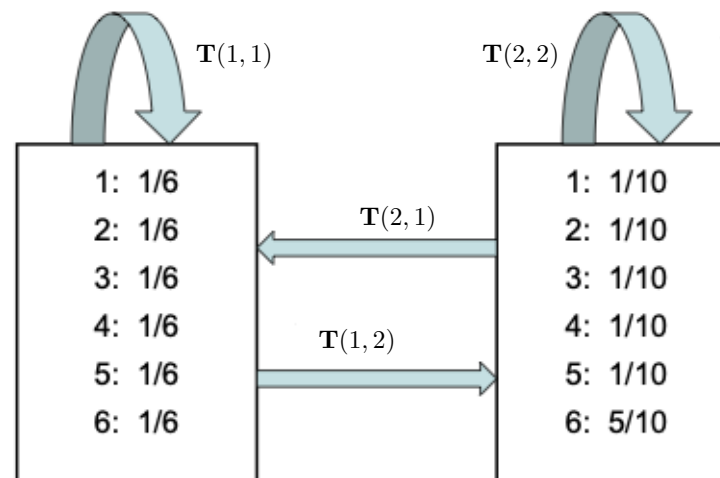


Applications

- Do not assume Markov property for the observations $\mathbf{x}[0], \mathbf{x}[1], \dots$. Allows **long-range dependencies** between observations captured through the latent variables $z[0], z[1], \dots$.
- **Speech recognition:** $\mathbf{x}[t]$ is the feature vector extracted from a speech signal and z_t is the word being spoken. The transition model $p(z[t] | z[t-1])$ represents the language model, while $p(\mathbf{x}[t] | z[t])$ represents the acoustic model.
- **Activity recognition:** $\mathbf{x}[t]$ is the feature vector extracted from an image or video frame and $z[t]$ is the type of activity a person is performing.
- **Gene finding:** $\mathbf{x}[t]$ represents the DNA nucleotides (A, C, G, T) and $z[t]$ represents whether we are inside a gene-coding region or not.
- Emission probabilities example: If $z[t] = k$, the observation $\mathbf{x}[t] \sim p(\mathbf{x}[t] | \phi_k) = \mathcal{N}(\mathbf{x}[t] | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

Toy Example

- Imagine you are playing dice. You suspect that one of the players is cheating.
- Most of the time, they use a standard die in which all six faces are equally probable, but every so often they secretly switch to a weighted die that comes up 6 half the time.
- To avoid getting caught, they don't switch dice very often: once they switch, they tend to keep using the same die for a while.
- If the player is rolling the fair die, they continue to do so with high probability, but they switch to the loaded die with a small probability. Similarly when rolling the loaded die.



Toy Example

We use a HMM to model this:

- Discrete latent states $z[t] \in \{1, 2\}$ to specify which of the two dice is used on the t -th roll. 1: fair die, 2: loaded die.
- You observe the outcomes or emissions $x[t] \in \{1, \dots, 6\}$.
- Latent state initial pdf π . What's the probability of starting with the fair or loaded die?
- Latent state transition matrix $\mathbf{T}$. What's the probability of switching from the fair to the loaded die, or vice versa?
- Emission probabilities (ϕ_1, ϕ_2) . What's the probability of rolling a number given the fair or loaded die?
 - ▶ $\phi_1 = (1/6, 1/6, 1/6, 1/6, 1/6, 1/6)$
 - ▶ $\phi_2 = (1/10, 1/10, 1/10, 1/10, 1/10, 5/10)$

EM for HMM: Baum-Welch Algorithm

- Suppose our training data $\mathcal{D} = \{\mathbf{x}_1[0:t_1], \dots, \mathbf{x}_n[0:t_n]\}$, where each $\mathbf{x}_i[0:t_i] = (\mathbf{x}_i[0], \mathbf{x}_i[1], \dots, \mathbf{x}_i[t_i])$ is a sequence of observations. Sequences are i.i.d. from each other. Note each $\mathbf{x}_i[t]$ can be a feature vector.
- If we observe the hidden states $\mathbf{z}_i = (z_i[0], z_i[1], \dots, z_i[t_i])$, we can just use vanilla MLE to estimate $\boldsymbol{\theta} = (\boldsymbol{\pi}, \mathbf{T}, \boldsymbol{\phi})$ from the likelihood below. (Exercise)

$$\begin{aligned} \log p(\mathcal{D} \mid \boldsymbol{\theta}) = & \sum_{i=1}^n \log \boldsymbol{\pi}(z_i[0]) + \sum_{i=1}^n \sum_{t=1}^{t_i} \log \mathbf{T}(z_i[t-1], z_i[t]) \\ & + \sum_{i=1}^n \sum_{t=0}^{t_i} \log p(\mathbf{x}_i[t] \mid \boldsymbol{\phi}_{z_i[t]}) \end{aligned}$$

- But now $z_i[t]$ is unknown. EM algorithm to the rescue!
- Also known as the Baum-Welch algorithm.

Baum-Welch: E step (Supplementary)

$$\begin{aligned}
 Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(m)}) &= \sum_{i=1}^n \mathbb{E}_{\boldsymbol{\theta}^{(m)}} \left[\log p(\mathbf{x}_i[0 : t_i], \mathbf{z}_i) \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)} \right] \\
 &= \sum_{i=1}^n \mathbb{E}_{\boldsymbol{\theta}^{(m)}} \left[\log \boldsymbol{\pi}(z_i[0]) \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)} \right] \\
 &\quad + \sum_{i=1}^n \sum_{t=1}^{t_i} \mathbb{E}_{\boldsymbol{\theta}^{(m)}} \left[\log \mathbf{T}(z_i[t-1], z_i[t]) \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)} \right] \\
 &\quad + \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbb{E}_{\boldsymbol{\theta}^{(m)}} \left[\log p(\mathbf{x}_i[t] \mid \boldsymbol{\phi}_{z_i[t]}) \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)} \right]
 \end{aligned}$$

Baum-Welch: E step (Supplementary)

$$\begin{aligned}
 &= \sum_{i=1}^n \sum_{z=1}^M \underbrace{p(z_i[0] = z \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})}_{\gamma_{i,0}(z)} \log \boldsymbol{\pi}(z) \\
 &\quad + \sum_{i=1}^n \sum_{t=1}^{t_i} \sum_{z, z'=1}^M \underbrace{p(z_i[t-1] = z, z_i[t] = z' \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})}_{\xi_{i,t}(z, z')} \log \mathbf{T}(z, z') \\
 &\quad + \sum_{i=1}^n \sum_{t=0}^{t_i} \sum_{z=1}^M \underbrace{p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})}_{\gamma_{i,t}(z)} \log p(\mathbf{x}_i[t] \mid \phi_z)
 \end{aligned}$$

Baum-Welch: E step (Supplementary)

- We need to compute the following quantities efficiently:
 - ▶ $\gamma_{i,t}(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})$
 - ▶ $\xi_{i,t}(z, z') = p(z_i[t-1] = z, z_i[t] = z' \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})$
- This is done by the **forward-backward** algorithm (details are provided as a supplementary).

Baum-Welch: M step (Supplementary)

- Assuming that the E step has been completed using the forward-backward algorithm to compute $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(m)})$. To maximize this, we proceed as usual by taking partial derivatives and solving for the root:

$$\hat{\boldsymbol{\pi}}(z) = \frac{\sum_{i=1}^n \gamma_{i,0}(z)}{n}, \quad (4.1)$$

$$\hat{\mathbf{T}}(z, z') = \frac{\sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, z')}{\sum_{u=1}^M \sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, u)}. \quad (4.2)$$

- To maximize w.r.t. ϕ , parameters of the emission probabilities $p(\mathbf{x} \mid \phi_z)$, depends on the specific model. E.g., Gaussian: $p(\mathbf{x} \mid \phi_z) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$ and the estimates are

$$\hat{\boldsymbol{\mu}}_z = \frac{\sum_{i=1}^n \sum_{t=0}^{t_i} \gamma_{i,t}(z) \mathbf{x}_i[t]}{\sum_{i=1}^n \sum_{t=0}^{t_i} \gamma_{i,t}(z)}, \quad (4.3)$$

$$\hat{\boldsymbol{\Sigma}}_z = \frac{\sum_{i=1}^n \sum_{t=0}^{t_i} \gamma_{i,t}(z) (\mathbf{x}_i[t] - \hat{\boldsymbol{\mu}}_z)(\mathbf{x}_i[t] - \hat{\boldsymbol{\mu}}_z)^\top}{\sum_{i=1}^n \sum_{t=0}^{t_i} \gamma_{i,t}(z)}. \quad (4.4)$$

Forward-Backward Algorithm (Supplementary)

- We want to compute $\gamma_{i,t}(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i], \boldsymbol{\theta}^{(m)})$. [Let's drop $\boldsymbol{\theta}^{(m)}$.]
- Basic idea: At time t , break the dependence on $\mathbf{x}_i[0 : t_i]$ into the “past” $\mathbf{x}_i[0 : t]$ and the “future” $\mathbf{x}_i[t + 1 : t_i]$:

$$\begin{aligned}
 \gamma_{i,t}(z) &= p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i]) \\
 &\propto p(z_i[t] = z, \mathbf{x}_i[t + 1 : t_i] \mid \mathbf{x}_i[0 : t]) \\
 &= p(z_i[t] = z \mid \mathbf{x}_i[0 : t]) p(\mathbf{x}_i[t + 1 : t_i] \mid z_i[t] = z, \mathbf{x}_i[0 : t]) \\
 &= p(z_i[t] = z \mid \mathbf{x}_i[0 : t]) p(\mathbf{x}_i[t + 1 : t_i] \mid z_i[t] = z), \tag{4.5}
 \end{aligned}$$

due to the Markov property.

- Let

$$\text{Forward } \alpha_i(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t]), \tag{4.6}$$

$$\text{Backward } \beta_i(z) = p(\mathbf{x}_i[t + 1 : t_i] \mid z_i[t] = z), \tag{4.7}$$

$$\text{Forward-Backward } \gamma_{i,t}(z) = \frac{\alpha_i(z) \beta_i(z)}{\sum_{z'} \alpha_i(z') \beta_i(z')} \propto \alpha_i(z) \beta_i(z).$$

Forward-Backward Algorithm: Forward (Supplementary)

- Consists of two recursive steps:

1. Predict:

$$\begin{aligned}
 & p(z_i[t] = z \mid \mathbf{x}_i[0 : t - 1]) \\
 &= \sum_{z'} \underbrace{p(z_i[t] = z \mid z_i[t - 1] = z')}_{\mathbf{T}(z', z) \text{ from } \boldsymbol{\theta}^{(m)}} \underbrace{p(z_i[t - 1] = z' \mid \mathbf{x}_i[0 : t - 1])}_{\alpha_{t-1}(z')},
 \end{aligned}$$

2. Update:

$$\begin{aligned}
 \alpha_t(z) &= p(z_i[t] = z \mid \mathbf{x}_i[0 : t]) \\
 &= p(z_i[t] = z \mid \mathbf{x}_i[t], \mathbf{x}_i[0 : t - 1]) \\
 &= \frac{1}{Z_i} \underbrace{p(\mathbf{x}_i[t] \mid z_i[t] = z)}_{\substack{\text{emission prob.} \\ \text{from } \boldsymbol{\theta}^{(m)}}} p(z_i[t] = z \mid \mathbf{x}_i[0 : t - 1]),
 \end{aligned}$$

where

$$Z_i = p(\mathbf{x}_i[t] \mid \mathbf{x}_i[0 : t - 1]) = \sum_z p(\mathbf{x}_i[t] \mid z_i[t] = z) p(z_i[t] = z \mid \mathbf{x}_i[0 : t - 1]).$$

- Initialize: $\alpha_0(z) = \frac{1}{Z_0} p(\mathbf{x}_i[0] \mid z_i[0] = z) \boldsymbol{\pi}(z)$, $Z_0 = \sum_z p(\mathbf{x}_i[0] \mid z_i[0] = z) \boldsymbol{\pi}(z)$.

Forward-Backward Algorithm: Backward (Supplementary)

- Recursively backwards from $\beta_{t+1}(\cdot)$:

$$\begin{aligned}
 \beta_t(z) &= p(\mathbf{x}_i[t+1:t_i] \mid z_i[t] = z) \\
 &= \sum_{z'} p(z_i[t+1] = z', \mathbf{x}_i[t+1], \mathbf{x}_i[t+2:t_i] \mid z_i[t] = z) \\
 &= \sum_{z'} \underbrace{p(z_i[t+1] = z' \mid z_i[t] = z)}_{\mathbf{T}(z, z')} \underbrace{p(\mathbf{x}_i[t+1] \mid z_i[t+1] = z)}_{\text{emission prob.}} \underbrace{p(\mathbf{x}_i[t+2:t_i] \mid z_i[t+1] = z')}_{\beta_{t+1}(z')}.
 \end{aligned}$$

- Initialize $\beta_{t_i}(z) = 1$. (Why? See (4.5).)

Conditional State Transition Probabilities (Supplementary)

- To compute $\xi_{i,t}(z, z') = p(z_i[t-1] = z, z_i[t] = z' \mid \mathbf{x}_i[0:t_i], \boldsymbol{\theta}^{(m)})$:

$$\begin{aligned}
 & \xi_{i,t}(z, z') \\
 & \propto p(\mathbf{x}_i[0:t_i] \mid z_i[t-1] = z, z_i[t] = z') p(z_i[t-1] = z, z_i[t] = z') \\
 & = p(\mathbf{x}_i[0:t-1] \mid z_i[t-1] = z) p(\mathbf{x}_i[t] \mid z_i[t] = z') p(\mathbf{x}_i[t+1:t_i] \mid z_i[t] = z') \\
 & \quad \cdot p(z_i[t] = z' \mid z_i[t-1] = z) p(z_i[t-1] = z) \\
 & \propto p(z_i[t-1] = z \mid \mathbf{x}_i[0:t-1]) p(\mathbf{x}_i[t] \mid z_i[t] = z') p(\mathbf{x}_i[t+1:t_i] \mid z_i[t] = z') \\
 & \quad \cdot p(z_i[t] = z' \mid z_i[t-1] = z) \\
 & = \alpha_{t-1}(z) \underbrace{p(\mathbf{x}_i[t] \mid z_i[t] = z')}_{\text{emission prob.}} \beta_t(z') \underbrace{p(z_i[t] = z' \mid z_i[t-1] = z)}_{\mathbf{T}(z, z')}.
 \end{aligned}$$

Summary: Training Using Baum-Welch Algorithm

1. Initialize $\theta^{(0)}$.
2. **E step:** At iteration m , use Forward-Backward Algorithm to compute:

Forward $\alpha_i(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t]),$

Backward $\beta_i(z) = p(\mathbf{x}_i[t + 1 : t_i] \mid z_i[t] = z),$

Forward-Backward $\gamma_{i,t}(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i], \theta^{(m)}) \propto \alpha_j(z)\beta_j(z)$

$$\xi_{i,t}(z, z') = p(z_i[t-1] = z, z_i[t] = z' \mid \mathbf{x}_i[0 : t_i], \theta^{(m)}) \\ \propto \alpha_{t-1}(z)p(\mathbf{x}_i[t] \mid z_i[t] = z')\beta_t(z')p(z_i[t] = z' \mid z_i[t-1] = z).$$

3. **M step:** Find $(m + 1)$ th guess of θ as

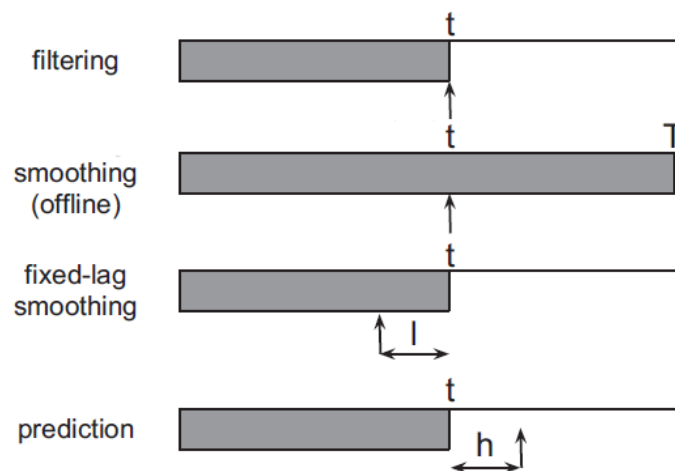
$$\hat{\pi}(z) = \frac{\sum_{i=1}^n \gamma_{i,0}(z)}{n},$$

$$\hat{\mathbf{T}}(z, z') = \frac{\sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, z')}{\sum_u \sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, u)},$$

$$\hat{\phi}_z = \text{emission prob. model parameters}$$

4. Repeat E and M steps.

Inference in HMMs



- **Filtering**: infer the latent state by computing $p(z[t] \mid \mathbf{x}[0 : t])$, i.e., make use of all observations $\mathbf{x}[0 : t]$ up to current time t - **Forward algorithm**.
- **Smoothing**: offline inference of latent state by computing $p(z[t] \mid \mathbf{x}[0 : T])$, i.e., make use of both past and future observations - **Forward-Backward algorithm**.
- **Fixed lag smoothing**: online inference by computing $p(z[t - l] \mid \mathbf{x}[0 : t])$, i.e., wait for a lag l before inferring latent state $z[t - l]$ - **Forward-Backward algorithm**.

Inference in HMMs

- **Prediction:** predict the future latent state by computing

$$\begin{aligned}
 p(z[t+h] \mid \mathbf{x}[0:t]) &= \sum_{z[t], \dots, z[t+h-1]} p(z[t], z[t+1], \dots, z[t+h-1], z[t+h] \mid \mathbf{x}[0:t]) \\
 &= \sum_{z[t], \dots, z[t+h-1]} \overbrace{p(z[t+h] \mid z[t+h-1]) p(z[t+h-1] \mid z[t+h-2]) \cdots p(z[t+1] \mid z[t])}^{\text{transition}} \\
 &\quad \cdot \underbrace{p(z[t] \mid \mathbf{x}[0:t])}_{\text{filtering}},
 \end{aligned}$$

where $h > 0$ is the prediction horizon. Can also use

$$p(\mathbf{x}[t+h] \mid \mathbf{x}[0:t]) = \sum_{z[t+h]} p(\mathbf{x}[t+h] \mid z[t+h]) p(z[t+h] \mid \mathbf{x}[0:t])$$

to infer the future observation $\mathbf{x}[t+h]$ (time series forecasting).

- **MAP** sequence: most probable sequence $\mathbf{z}^*[0:T] = \arg \max_{\mathbf{z}[0:T]} p(\mathbf{z}[0:T] \mid \mathbf{x}[0:T])$ - **Viterbi algorithm**.

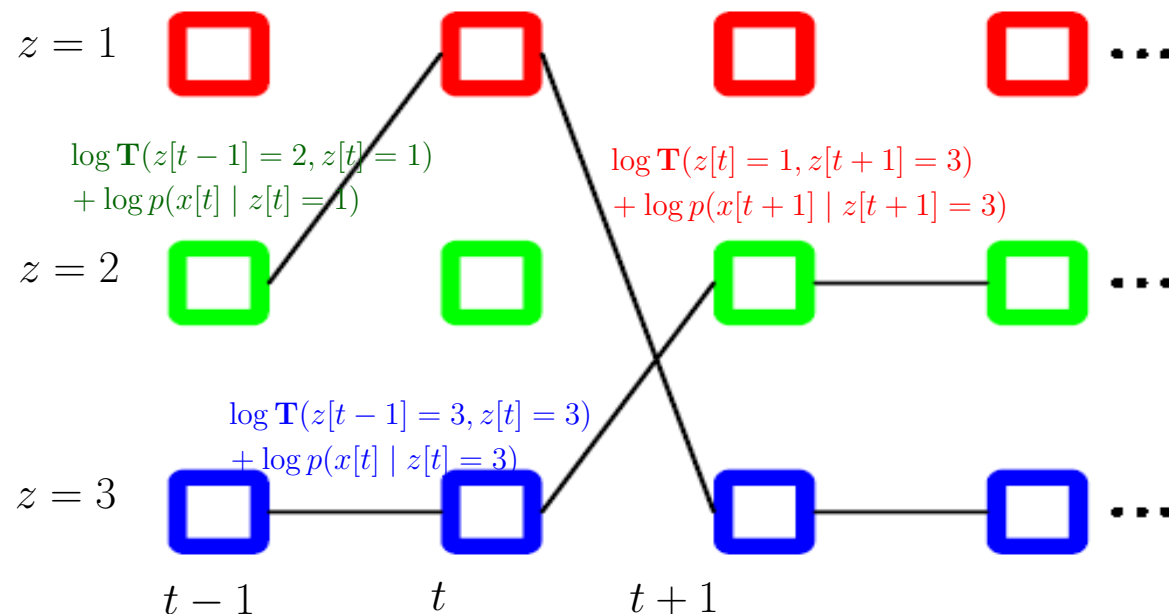
Viterbi Algorithm (Supplementary)

- MAP estimate $\mathbf{z}[0 : T]^* = \arg \max_{\mathbf{z}[0:T]} p(\mathbf{z}[0 : T] \mid \mathbf{x}[0 : T])$, most probable sequence of latent states given the observations $\mathbf{x}[0 : T]$. Find sequence or *path* $z[0], z[1], \dots, z[T]$ to maximize

$$\log p(\mathbf{z}[0 : T] \mid \mathbf{x}[0 : T])$$

$$= \log p(\mathbf{z}[0 : T], \mathbf{x}[0 : T]) + \text{const}$$

$$= \log \boldsymbol{\pi}(z[0]) + \log p(x[0] \mid z[0]) + \sum_{t=1}^T (\log \mathbf{T}(z[t-1], z[t]) + \log p(x[t] \mid z[t])).$$



Viterbi Algorithm (Supplementary)

- At time t , let

$$\delta_t(z) = \max_{\mathbf{z}[0:t-1]} \log p(\mathbf{z}[0:t-1], z[t] = z, \mathbf{x}[0:t]),$$

the log probability of the most likely path to end up at state $z[t] = z$ at time t .

- Suppose this most likely path is $(z[0], z[1], \dots, z[t-1] = z', z[t] = z)$.
- Then $(z[0], z[1], \dots, z[t-1] = z')$ must be a most likely path to reach state $z[t-1] = z'$.
- Because otherwise, we can choose a most likely path $(\tilde{z}[0], \tilde{z}[1], \dots, \tilde{z}[t-1] = z')$ to reach z' and then transition to $z[t] = z$.

Viterbi Algorithm (Supplementary)

$$\begin{aligned}
 \delta_t(z) &= \max_{\mathbf{z}[0:t-1]} \log p(\mathbf{z}[0:t-1], z[t] = z, \mathbf{x}[0:t]) \\
 &= \max_{\mathbf{z}[0:t-1]} \{ \log p(\mathbf{z}[0:t-1], \mathbf{x}[0:t-1]) + \log p(z[t] = z \mid \mathbf{z}[0:t-1]) + \log p(\mathbf{x}[t] \mid z[t] = z) \} \\
 &= \max_{\mathbf{z}[0:t-2], z[t-1]=z'} \{ \log p(\mathbf{z}[0:t-1], \mathbf{x}[0:t-1]) + \log p(z[t] = z \mid z[t-1] = z') \\
 &\quad + \log p(\mathbf{x}[t] \mid z[t] = z) \} \\
 &= \max_{z'} \{ \delta_{t-1}(z') + \log \mathbf{T}(z', z) \} + \log p(\mathbf{x}[t] \mid z[t] = z)
 \end{aligned}$$

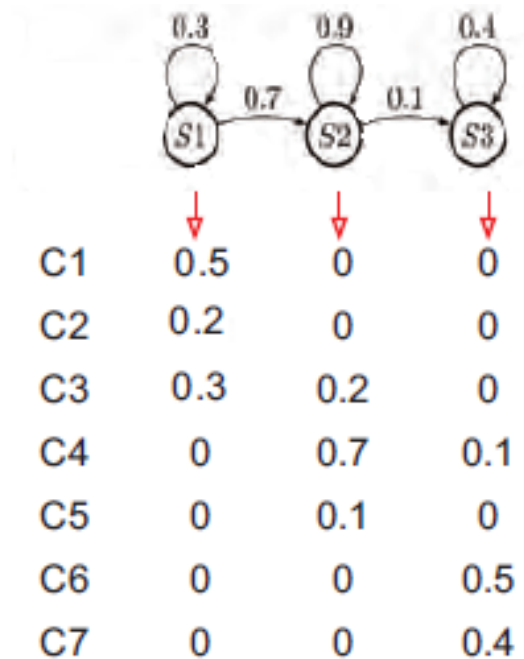
- Initialize $\delta_0(z) = \log \boldsymbol{\pi}(z) + \log p(\mathbf{x}[0] \mid z[0] = z)$.
- For each $t = 1, \dots, T$, for each $z = 1, \dots, M$, previous state on most likely path to $z[t] = z$ is

$$a_t(z) = \arg \max_{z'} \{ \delta_{t-1}(z') + \log \mathbf{T}(z', z) \}.$$

Compute $\delta_t(z) = \delta_{t-1}(a_t(z)) + \log \mathbf{T}(a_t(z), z) + \log p(\mathbf{x}[t] \mid z[t] = z)$.

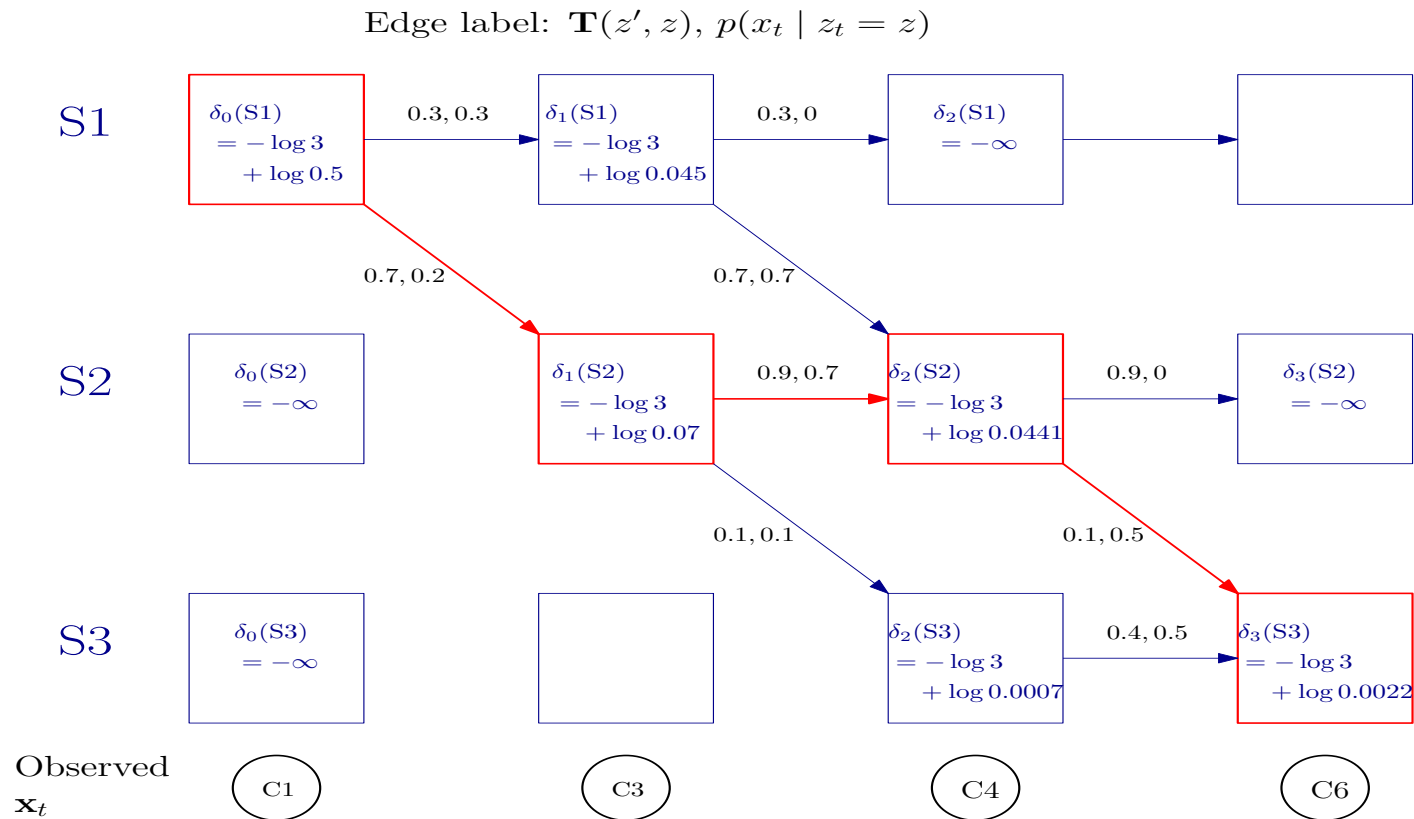
- Terminate: final state on most probable path is $z^*[T] = \arg \max_z \delta_T(z)$.
- Traceback: for $t = T - 1, \dots, 1, 0$, $z^*[t] = a_{t+1}(z^*[t+1])$.

Viterbi Algorithm Example (Supplementary)



- $z_t \in \{S1, S2, S3\}$, $\mathbf{x}_t \in \{C1, C2, \dots, C7\}$. Figure shows the emission probabilities. Suppose $\pi(z) = 1/3$ for all z .
- Suppose we observe $\mathbf{x}[0 : 3] = (C1, C3, C4, C6)$.

Viterbi Algorithm Example (Supplementary)



- $\delta_0(S1) = -\log 3 + \log 0.5$, $\delta_0(z) = -\infty$ for $z = S2, S3$ because $p(\mathbf{x}[0] = C1 | z_0 = z) = 0$.
- S1 transitions to S1 with probability $\mathbf{T}(S1, S1) = 0.3$ and $p(\mathbf{x}[1] = C3 | z_1 = S1) = 0.3$.

$$\delta_1(S1) = \delta_0(S1) + \log 0.3 + \log 0.3 = -\log 3 + \log 0.045.$$

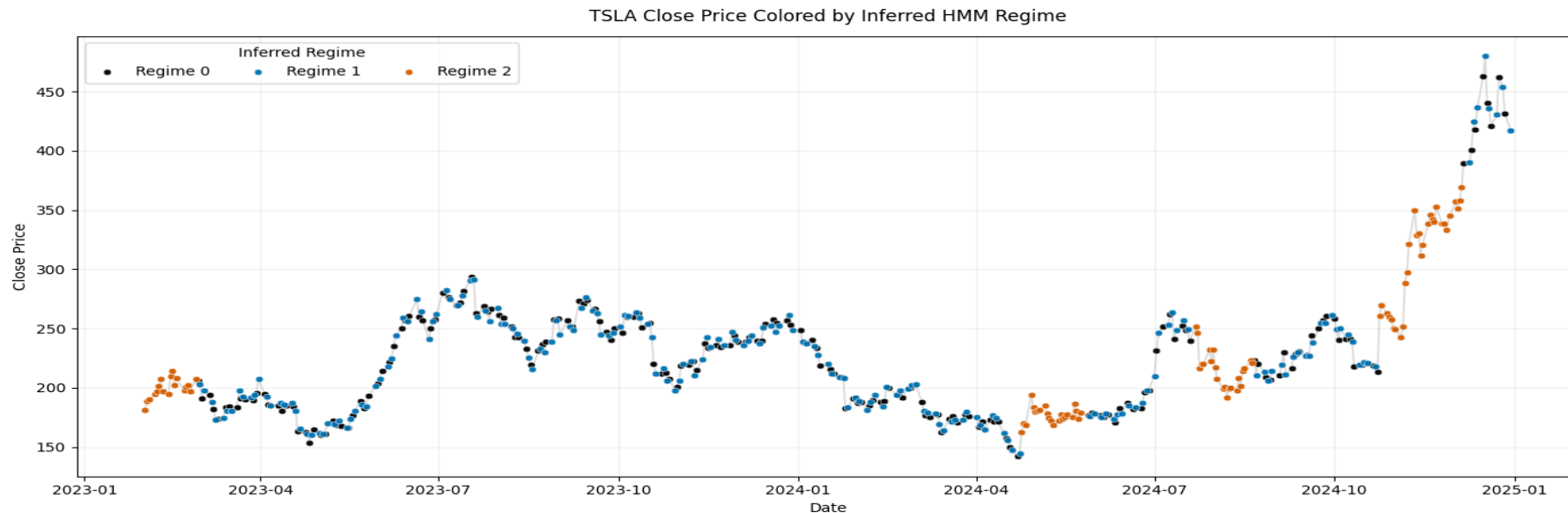
- S1 transitions to S2 with probability $\mathbf{T}(S1, S2) = 0.7$ and $p(\mathbf{x}[1] = C3 | z_1 = S2) = 0.2$.

$$\delta_1(S2) = \delta_0(S1) + \log 0.7 + \log 0.2 = -\log 3 + \log 0.07.$$

- Exercise: check the rest!

A Real Application in Finance

- [04_finance_hmm.ipynb](#)
- Modeling the price of a stock as a HMM with three states (regimes).
- Observations are the daily log returns and volatility of a stock ticker.



Summary

- Properties of Markov chains.
- MLE for Markov model.
- Baum-Welch algorithm for HMM.
 - ▶ E step: forward-backward algorithm.
 - ▶ M step: maximize for θ .
- Inference in HMM: Viterbi algorithm to find the most likely state path.

Outline

- 1 Markov Chains
- 2 Hidden Markov Models
- 3 Some Practice Problems
- 4 MLE Derivation (Supplementary)

Some Practice Problems

A wearable ring sensor as shown in the following figure can be used to track the movement of a finger. The sensor can be in one of three states: still (S), moving up (U), and moving down (D). An inertial measurement unit (IMU) embedded in the ring measures the acceleration of the finger movement in three dimensional space. The IMU data is transmitted wirelessly to a computer for processing.



- a) The IMU measurement is a three dimensional vector $\mathbf{x} = (x_1, x_2, x_3)$, where x_1, x_2, x_3 are the accelerations in three fixed directions, respectively. Suppose the IMU data is modeled as a Gaussian mixture model (GMM) with three components corresponding to the sensor state $z \in \{S, U, D\}$. Write down the GMM.
- b) Should a Markov chain or a hidden Markov model (HMM) be used at the processing computer to model the sensor state? Explain your answer.

Some Practice Problems (cont.)

- c) Suppose you decide to use a HMM. The sensor can be in the S state for a long time but not in the other states. The sensor can transition from S to U or D , but it can only transition from U to D or from D to U . Draw the transition diagram of the HMM.
- d) The Baum-Welch algorithm is used to estimate the parameters of the HMM. What are the parameters to be estimated?
- e) Given a sequence of IMU measurements $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, what algorithm would you use to find the most likely sequence of sensor states $z_1, z_2, \dots, z_n$?

Solution:

a)

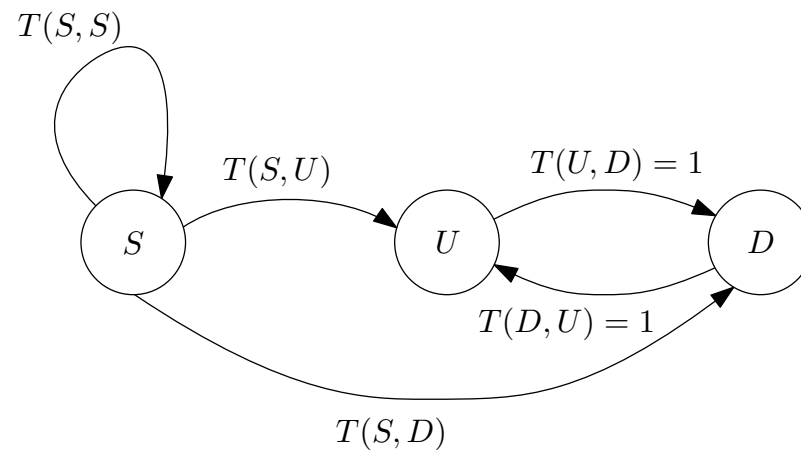
$$\begin{aligned}
 p(\mathbf{x}) &= \sum_{z \in \{S, U, D\}} \pi(z) p(\mathbf{x} | z) \\
 &= \sum_{z \in \{S, U, D\}} \pi(z) \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z),
 \end{aligned}$$

Some Practice Problems (cont.)

where $\pi(z)$ is the prior probability of the sensor state z and μ_z, Σ_z are the mean and covariance matrix of the IMU measurement $\mathbf{x}$ in state z .

b) A HMM should be used because the sensor state is not directly observable at the processing computer.

c)



d) The parameters to be estimated are the prior probabilities $\pi(z)$, the transition probabilities $p(z' | z)$, and the mean μ_z and covariance matrix Σ_z of the IMU measurement $\mathbf{x}$ in each state z .

e) The Viterbi algorithm is the optimal algorithm to find the most likely sequence of sensor states $z_1, z_2, \dots, z_n$.

Outline

- 1 Markov Chains
- 2 Hidden Markov Models
- 3 Some Practice Problems
- 4 MLE Derivation (Supplementary)

MLE Derivation (Supplementary)

1. Derive the MLE of $\theta = (\pi, \mathbf{T}, \phi)$ for a HMM when the training data $\mathcal{D} = \{(\mathbf{x}_1[0:t_1], \mathbf{z}_1[0:t_1]), \dots, (\mathbf{x}_n[0:t_n], \mathbf{z}_n[0:t_n])\}$ (i.e., the latent states are observed). Assume that the emission probabilities are Gaussian distributions.

Solution:

$$\begin{aligned} \log p(\mathcal{D} \mid \theta) &= \sum_{i=1}^n \log \pi(z_i[0]) + \sum_{i=1}^n \sum_{t=1}^{t_i} \log \mathbf{T}(z_i[t-1], z_i[t]) \\ &\quad + \sum_{i=1}^n \sum_{t=0}^{t_i} p(\mathbf{x}_i[t] \mid \phi_{z_i[t]}) \end{aligned}$$

Maximizing the Lagrangian $\sum_{i=1}^n \log \pi(z_i[0]) - \lambda(\sum_{z'} \pi(z') - 1)$ w.r.t. $\pi(z)$, we obtain

$$\hat{\pi}(z) = \frac{N_z^0}{n}, \quad N_z^0 = \sum_{i=1}^n \mathbf{1}\{z_i[0] = z\}. \quad (4.8)$$

MLE Derivation (Supplementary) (cont.)

Maximizing the Lagrangian

$\sum_{i=1}^n \sum_{t=1}^{t_i} \log \mathbf{T}(z_i[t-1]z_i[t]) - \lambda(\sum_{u=1}^M \mathbf{T}(z, u) - 1)$ w.r.t. $\mathbf{T}(z, z')$ and finding the root, we obtain

$$\hat{\mathbf{T}}(z, z') = \frac{N_{zz'}}{\sum_{u=1}^M N_{zu}}, \quad N_{zu} = \sum_{i=1}^n \sum_{t=1}^{t_i} \mathbf{1}\{z_i[t-1] = z, z_i[t] = u\}. \quad (4.9)$$

For Gaussian emission probabilities, we have $p(\mathbf{x} \mid \phi_z) = \mathcal{N}(\mathbf{x} \mid \mu_z, \Sigma_z)$. We thus have

$$\begin{aligned} & \sum_{i=1}^n \sum_{t=0}^{t_i} \log p(\mathbf{x}_i[t] \mid \phi_{z_i[t]}) \\ &= -\frac{1}{2} \sum_{i=1}^n \sum_{t=0}^{t_i} \sum_{z=1}^M \mathbf{1}\{z_i[t] = z\} \log |\Sigma_z| \\ & \quad - \frac{1}{2} \sum_{i=1}^n \sum_{t=0}^{t_i} \sum_{z=1}^M \mathbf{1}\{z_i[t] = z\} (\mathbf{x}_i[t] - \mu_z)^\top \Sigma_z^{-1} (\mathbf{x}_i[t] - \mu_z). \end{aligned}$$

MLE Derivation (Supplementary) (cont.)

Maximizing w.r.t. $\boldsymbol{\mu}_z$, we have

$$\sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\} \boldsymbol{\Sigma}_z^{-1} (\mathbf{x}_i[t] - \boldsymbol{\mu}_z) = 0$$

$$\hat{\boldsymbol{\mu}}_z = \frac{\bar{\mathbf{x}}_z}{N_z}, \quad \bar{\mathbf{x}}_z = \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\} \mathbf{x}_i[t], \quad N_z = \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\}. \quad (4.10)$$

MLE Derivation (Supplementary) (cont.)

Maximizing w.r.t. Σ_z together with the constraint that $\Sigma \succ 0$,

$$\begin{aligned}
 & \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\} \Sigma_z^{-1} \\
 & + \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\} \Sigma_z^{-1} (\mathbf{x}_i[t] - \boldsymbol{\mu}_z)(\mathbf{x}_i[t] - \boldsymbol{\mu}_z)^\top \Sigma_z^{-1} = 0 \\
 \hat{\Sigma}_z &= \frac{1}{N_z} \sum_{i=1}^n \sum_{t=0}^{t_i} \mathbf{1}\{z_i[t] = z\} (\mathbf{x}_i[t] - \boldsymbol{\mu}_z)(\mathbf{x}_i[t] - \boldsymbol{\mu}_z)^\top. \tag{4.11}
 \end{aligned}$$

MLE Derivation (Supplementary) (cont.)

2. Carefully verify (4.1) - (4.4).

Solution:

$$\begin{aligned}
 Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(m)}) &= \sum_{i=1}^n \sum_{z=1}^M \gamma_{i,0}(z) \log \boldsymbol{\pi}(z) + \sum_{i=1}^n \sum_{t=1}^{t_i} \sum_{z, z'=1}^M \xi_i[t](z, z') \log \mathbf{T}(z, z') \\
 &+ \sum_{i=1}^n \sum_{t=0}^{t_i} \sum_{z=1}^M \gamma_i[t](z) \log p(\mathbf{x}_i[t] \mid \phi_z).
 \end{aligned}$$

Maximizing w.r.t. $\boldsymbol{\pi}(z)$ with the constraint $\sum_z \boldsymbol{\pi}(z) = 1$, we obtain (replace $\mathbf{1}\{z_{i,0} = z\}$ in (4.8) with $\gamma_{i,0}(z)$)

$$\hat{\pi}(z) = \frac{\sum_{i=1}^n \gamma_{i,0}(z)}{n}.$$

MLE Derivation (Supplementary) (cont.)

Maximizing w.r.t. $\mathbf{T}(z, z')$, we obtain (replace $\mathbf{1}\{z_i[t-1] = z, z_i[t] = u\}$ in (4.9) with $\xi_i[t](z, u)$)

$$\hat{\mathbf{T}}(z, z') = \frac{\sum_{i=1}^n \sum_{t=1}^{t_i} \xi_i[t](z, z')}{\sum_{u=1}^M \sum_{i=1}^n \sum_{t=1}^{t_i} \xi_i[t](z, u)}.$$

Equations (4.3) and (4.4) are obtained in the same way.